ECE 350: Real-Time Operating Systems Winter 2027

Lecture 33 — Trusted Computing

Jeff Zarnett *2026-09-01*

Whose PC is it Anyway?

In the discussion of kernel modules, we learned that online multiplayer games use kernel-level anti-cheat mechanisms. Even that is sometimes not sufficient from the view of game developers. In 2025, for example, Activision/Blizzard announced that they would require TPM 2.0 and Secure Boot to play Call of Duty. These two things are hardware-based functionality that are ostensibly about security, though as we will see, things are slightly more complex than that, because these security features open questions about whom we are trying to secure the computer from, and why.

The focus on hardware-level protection of what is allowed to run on the computer started around the year 2000, with the publication of the version 1.0 specification of Trusted Computing. It is *not at all* a coincidence that this conversation started at a time where the hot topic amongst the megacorporations was "stopping music piracy".

History lesson on this: in June of 1999, the application Napster launched, which was a peer-to-peer music sharing service that let people download copies of mp3 music files shared by other users, without paying. This made a number of big media conglomerates 10 out of 10 angry, because they argued that every downloaded (pirated) copy was a lost sale and therefore they were losing millions upon millions of dollars. I personally don't think this is true, but there were a lot of lawsuits and things got ugly. The big players were looking to stop this through something called Digital Rights Management (DRM): the ability to stop you from doing things they do not want you to do with the media that you purchased.

Digital Rights Management is a little euphemistic, because it is really digital restrictions management. These restrictions only work if the software understands what you are and are not allowed to do, and enforces those constraints. So perhaps the video player permits you to view the video but not to save it, or the reader software permits you to read an e-book but not take a screenshot. You can very likely think of some workarounds for these restrictions immediately, if you control the computer and its software.

Before I say anything concrete about that, let me stop for a quick legal disclaimer. Circumvention of DRM restrictions is a legal minefield: there exist valid reasons and circumstances where it is your legal right to do so, and other circumstances where the same actions are a violation of copyright law. We can talk about how it works, technically speaking, but I do not have the legal expertise to tell you what's permitted and what's not in any jurisdiction so... consult a lawyer to learn the specifics and get qualified advice, if you need.

For example, if the media player has the download function but it's (sometimes) disabled, you can just... edit the application binary so the if-else condition that determines if this button should be enabled always says it is enabled. If the media is encrypted, you can find the encryption key in the process memory and use that to decrypt the media. If there's some network call to check for authorization, you can redirect that call to a local server you control that always says yes. If the operating system forbids copying the file somehow, you can just override that with a kernel module that lets you do it. "Do whatever you want with your purchased content with this one weird trick! Corporations hate him!"

We Have to Go Deeper. Security, as discussed previously, is always an arms race. The key factor in all of the examples given above is that the person who purchased and administers the computer has a lot of ability to modify application software and even the OS. Each of the examples just given about how to defeat restriction measures is not trivial; most people probably do not have the knowledge or skills to do it. Still, if even one person creates an unencrypted copy of the movie and shares it, it can propagate and be watched by anybody. Which, as we know, the movie studios desperately want to stop. So the prevention of circumvention of DRM has to move lower down,

into hardware, because that's the only way to break that condition where the administrator of the computer has enough control to bypass restrictions.

To quote a simple explainer: "TC provides a computing platform on which you can't tamper with the application software, and where these applications can communicate securely with their authors and with each other. The original motivation was digital rights management (DRM): Disney will be able to sell you DVDs that will decrypt and run on a TC platform, but which you won't be able to copy. The music industry will be able to sell you music downloads that you won't be able to swap. They will be able to sell you CDs that you'll only be able to play three times, or only on your birthday." [And03].

It's not in the article, because that is from 2003 and a great many things have gotten worse now, but the current state of DRM is that it allows the corporation to alter the deal at any time. In 2026, for example, Sony deleted movies (including *Terminator 2: Judgment Day*) that customers paid for, removing them from their accounts with no refund or compensation [Wal26]. Yes, yes, they say things like "you are purchasing a *license* to watch the movie under limited circumstances" and there are more terms buried in the legalese about their right to do this when their licensing deal with the movie studio ends, but whether or not it is technically legal does not diminish the danger here: they *can* do this and the consumer has no way to stop it.

From what we already discussed, we can see why a hardware-level solution is necessary if the goal is to meaningfully enforce these restrictions. If I, as the user, control the kernel – up to and including live-patching it – I can bypass restrictions. If the computer will refuse to boot a kernel that is not "approved" by the authorities (and such an approved kernel would, naturally, forbid any sort of live-patching) it is much more difficult to work around the restrictions that the content companies (or others) want to impose[1].

There are further concerns outlined in [And03] that this could go beyond just preventing people from watching media or copying software, and that it can be used to delete content that the government, or other powerful actors, don't want you to have. The standard dystopian example of this: imagine a journalist has published an exposé on the corruption and criminality of the dictator of a country, and said dictator simply has the ability to order that all copies of that document are deleted from everyone's computer.

At least at present, if verification fails, the computer is not useless: it's just in an "untrusted" state, and that restricts access to certain things. This means that you can still boot your operating system, run some applications, and do lots of things. But not play the desired game or consume restricted media.

It is, however, not unthinkable that the future state is that it will be difficult to run the computer in an untrusted state at all[2]. Yes, it is likely that Linux and other open-source options will exist, but as life becomes increasingly conducted online, it gets harder and harder over time to use a "free" computer if you cannot use it to do the things needed in everyday life: edit Word/Google documents, use online banking, pay taxes, order online from popular merchants, etc. Will you need two computers? Oh probably.

It is also important to note that perfection on digital restrictions like this is impossible for multiple reasons. One of them is referred to as "the analog hole". No matter how restricted the playback of media is on the computer, for you to perceive it as a human, you will use your eyes and ears. That means the content is shown on a screen and the photons leave the monitor and travel to your eyes, and sound is vibrations in the air created by the speakers that travel to your ears. Those then become vulnerable to recording. This happens when – just as an example – people take cameras into movie theatres and record the movie when it's showing. The quality is bad compared to what would be available if streaming it, yet people are willing to do this and to watch the lower-quality version. Similarly, if the document cannot be copied by normal means, someone trying to circumvent that can take pictures of it, re-type it, or write it out by hand on paper. People will find a way; it's just a question of how hard it is.

With that said: under no circumstances should you let Elon Musk (or anybody else, but especially not him) put some chip in your brain to "permit" you to watch restricted content. That would, in theory, solve the analog hole problem, but the future where media companies can control your brain is much more corporate serfdom than it is Cyberpunk 2077. Just say no to brain implants, at least, the ones offered by megacorps.

[1] With that said, people almost always FIND a way.

[2] They're getting pretty close with phones and tablets.

Less-Corporate-Overlord Reasons. While I do think the primary motivation for the trusted computing platform really is to take the power out of your hands as the owner of the machine and let the megacorporations decide what you are "allowed" to do, there are practical, security-focused reasons for trusted computing and secure boot.

Gaming provides a legitimate example. If you want to be sure that the other players are not cheating, the kernel-level anti-cheat module can be backstopped by the trusted computing platform: if the kernel (and its extensions) are not all "approved", that means the anti-cheat module cannot guarantee the integrity of the system (because, potentially, some cheating kernel extension is loaded) and will therefore decline the request to start the game.

There are also scenarios where locking down documents may serve reasonable government or military purposes. For example, the application of the correct restrictions means it ceases to be possible (or is at least a lot harder) for people to share classified military documents on the forums of the game *War Thunder* to win an argument over how realistic the vehicles are in gameplay. You would think that this is a joke, but according to Wikipedia, it has happened (as of the time of writing) more than 15 times.

Another security benefit is defeating low-level compromises of the system that the OS itself may struggle to detect. If malware is installed in the system firmware or boot loader, the malware can survive if the OS is reinstalled. The OS cannot find it, because the OS is running itself on a compromised foundation. This is a *bootkit*. The name is a variation of the term `rootkit`, software with the purpose of giving root (administrator) access to someone who is not meant to have it. Rootkits run inside the operating system itself; a bootkit starts before that.

The supposed solution to that is called Secure Boot. The Arch Linux wiki[3] defines this as a security feature in the UEFI standard to verify these low-level foundational components to be sure they are valid. Only after verification can control be transferred to it, and the next stage can be assessed and verified.

Unsurprisingly, these mechanisms are not perfect either. There are numerous examples of bypassing secure boot. To give a specific example, consider BlackLotus, an exploit that was in the wild at least from 2022. This malware uses a known vulnerability to bypass secure boot, and exactly like the driver revocation problem we discussed, it uses signed binaries that have not been revoked even though they have a vulnerability. When installed, it can disable kernel security mechanisms, load a kernel driver, and then it infects the `winlogon.exe` process with some HTTP downloader that is used to permit bad actors to issue commands to that computer [Smo23]. That's scary stuff.

Access Denied. Alongside the idea of secure boot is *measured boot*. Instead of just allow or deny kind of semantics, measured boot hashes and records the state of the things that ran, whether that's been altered or not. This permits detecting a change – if you notice that the boot loader hash value no longer matches what it was before, and this change is unexpected, it is a potential warning sign of a compromise. This would permit detecting the BlackLotus rootkit.

The Arch Linux wiki[4] outlines that these measured boot values can also be used as a gating function for whether access is permitted. To do this, we need some hardware support: the Trusted Platform Module (TPM). The TPM permits management of encryption keys. Sometimes, the TPM is a discrete motherboard component and it is possible to snoop the encryption key by looking at the wires carrying the signal to the bus. In other cases, the TPM lives inside the CPU so that ceases to be an option.

Suppose, as in the example in the Arch Linux wiki, that the boot drive of the system is encrypted. The encryption key will be sealed by the hardware TPM, and the module will release the encryption key only if the assessment of the boot state is acceptable.

We understand, of course, that changes to these components are desirable. Boot loaders, firmware, and kernels all have bugs and upgrades. So the hash values of these will change, on purpose, so it cannot be the case that changing this is impossible. Attackers might attempt to use the change of expectation mechanism to make the compromised versions appear to be expected. Always an arms race, remember...

[3] `https://wiki.archlinux.org/title/Unified_Extensible_Firmware_Interface/Secure_Boot`
[4] `https://wiki.archlinux.org/title/Trusted_Platform_Module`

Attestation

We've discussed two ways in which the state of the system can be assessed: the game startup checks and the boot-up checks. We are, however, left with the question as to whether those checks produce reliable results. After all, if you ask a liar if they are lying, will they say yes? Malware is not going to say it's malware when asked. The solution is called *attestation*: verifying, externally, the claims.

Attestation follows a short sequence of steps. We start with the measurement: the evidence collected by the game launcher or the system at startup. The local system requests verification from the remote system; the remote system, called the challenger (or appraiser), sends a random one-time code that needs to be incorporated. The local machine takes this random code, combines it with the measurement, and then uses the TPM to sign it using the TPM's signing key. The local machine sends it over. The challenger can then validate the cryptographic signature is correct, validate the one-time code, and then can check the measurement. If all three parts are valid, the system is in an approved state [CGL+11].

To make a more concrete example: for the game, the state of the system is inspected: is secure boot on, is the TPM sufficiently advanced, did the system boot only using trusted components and approved modules? No unusual kernel modules or similar loaded? If this meets with the approval of the game server, then you can join the game and play. Okay, you can imagine that the server validates this not just when launching the game or even when joining a new match, because users could tamper with things in-between; it seems likely that validation is periodic or continuous. But the procedure described above can be repeated arbitrarily many times. If at any point the validation fails, forbid joining the game... or kick the user from the game if they're in the middle of one.

Architecturally speaking, we get around the problem of asking a system to self-report on its honesty, because there exists external validation of the state of the system. The validator is not under the control of the bad actors, as it is a remote system, so we can assume that it behaves reliably. While we often think of remote systems as being the ones that we do not trust, keep in mind we should view the threat model from the view of the game server, not the game client. The game servers running in whatever (cloud computing?) data centre view the unverified client (player computers) as the potential source of threat.

The one-time code prevents replaying an old response, and the cryptographic signing is hard to forge because only the machine's TPM has the (unique) signing key.

Validating the signing key is farther down the cryptography and security track than we consider to be in-scope for this course, but suffice it to say that the challenger has mechanisms to check that the signing key is valid and comes from an approved TPM source. There are also concerns about whether this signing key can uniquely identify a computer, limiting privacy and the answer to that is yes but with a workaround... also beyond the scope of this course.

We forgot to talk about one thing, though: how can we be sure that the measurement is valid? Remember that verification happens at each stage, before we go on to the next one. If we start from a secure place, like the UEFI, it will accurately record the boot loader's measurement. If the kernel is compromised, that compromise has already been recorded before it gets a chance to run. And the record cannot be rewritten by a malicious kernel.

That just moves the problem one step along: why can the kernel not rewrite the record? It has effectively unlimited access to the computer. Yes, but the TPM itself is the storage location for the measurements. And the TPM offers the capability to read, reset, and append to the record. Okay, append is not like a log file here, it actually involves combining the new hash with the old. So there's no way to go back and change something already written in this session. It's possible to start again with reset, but that just leads to the same behaviour. There simply is no hardware command that would permit changing the record so no malicious actor can activate it.

You might have also picked up that I used the phrase "if we start from a secure place". That condition is important, and leads to our next idea... the chain of trust problem.

The Chain of Trust Problem

All of this might make you feel a certain level of concern as to what exactly is going on in the devices that you allegedly own. That is a rational thing to be worried about, because we can see that there are so many points at which a bad actor could compromise the security of the system: the hardware, the boot loader, the kernel, the OS services, the compiler or interpreter, and the source code of the program. If any of those is compromised, then the confidential document you are reading can be leaked, and before you know it, fraudsters are taking out loans in your name.

Part of the argument for open-source software is that you can, at least in principle, audit the code in it and understand what it does and whether that matches what it says it would do. Analysis of closed-source software is not impossible, of course, but it is much harder. But any complicated software project is gigantic and not everyone has the skills to do that, let alone the time.

The problem is worse in the world of cloud computing. The previous topic introduced virtualization and yet this topic has ignored that entirely and discussed devices that you own. Cloud computing is rather diametrically opposed to that view: you own nothing, but instead rent some resources from the cloud provider. So you need to trust the cloud provider, its hardware, its hypervisor, or the people who run the cloud computing platform in general.

Near-Miss? `xz` and `ssh`. Early in 2024, a compromise was discovered in the `xz` library and Andres Freund posted about it on the mailing list [Fre24]. The library itself is used for data compression but some versions of `openssh` rely on it. This is a strong example of what's termed a "supply chain attack".

The term comes from the logistical supply chain. Supply chains for physical products are often complicated; if you want to build a gaming PC you need a large number of different components and you only get the end product when you've got all the pieces at hand. The problem is recursive, though: how many different components and pieces did, for example, NVIDIA require to make your graphics card? If one of the components—however small or seemingly-insignificant—is unavailable for whatever reason, the device (or computer) can't be built.

Similarly, almost any modern piece of software that isn't a toy or academic assignment is built in a compositional way (app A uses libraries B, C, D each of which has dependencies $E, F, G, H...$). Compromising any one of the dependencies F could result in a vulnerability in A. But if A is `openssh`, you know, the *secure* shell daemon, being able to exploit that means being able to have root access on the vulnerable server and execute arbitrary code.

And yet, it's worse than that. The compromise of the library was in the build system for `xz`, not the code itself, so merely examining the source code of this library would not have found the problem.

Bad Things or Bad Actors? Ken Thompson (one of the legends of Bell Labs) told us in a 1984 lecture that you cannot trust code that you did not write yourself or that's built using a toolchain that you did not write. Ultimately, it is thoroughly infeasible to verify everything from hardware to application. There's just too much to verify, and software is so complex it tends to defy thorough analysis. Ken would also tell you that you need to stop looking for "bad things" and ask yourself whether you can trust the authors of the code[5]. And we have to put some trust in *some people*, otherwise we will never be able to accomplish anything.

It feels bad, in a way, to say that we have to just trust other people (and their AI agents). Sure, but we have to trust in real life, too, because we live in a society: did you grow all your own food, cook it all yourself, wear only clothing you've sewn from fabric you've made, and built your own home with tools that you made yourself? Obviously, a battery-powered drill isn't going to leak your health records to hackers. But you are still counting on it to not explode in your hands or burn your home down, right? Trust is fundamental to society and being overly paranoid is not a solution either. Look skeptically at software you do not control or did not write... but do so proportionally with the magnitude of the risks.

[5] AI tools make it much harder to say yes to this question.

References

[And03] Ross Anderson. 'trusted computing' frequently asked questions, 2003. Online; accessed 16-August-2026. URL: https://www.cl.cam.ac.uk/archive/rja14/tcpa-faq.html.

[CGL+11] George Coker, Joshua Guttman, Peter Loscocco, Amy Herzog, Jonathan Millen, Brian O'Hanlon, John Ramsdell, Ariel Segall, Justin Sheehy, and Brian Sniffen. Principles of remote attestation. *International Journal of Information Security*, 10(2):63–81, 2011. URL: https://doi.org/10.1007/s10207-011-0124-7.

[Fre24] Andres Freund. backdoor in upstream xz/liblzma leading to ssh server compromise, 2024. Online; accessed 14-July-2024. URL: https://www.openwall.com/lists/oss-security/2024/03/29/4.

[Smo23] Martin Smolár. Blacklotus uefi bootkit: Myth confirmed, 2023. Online; accessed 23-August-2026. URL: https://www.welivesecurity.com/2023/03/01/blacklotus-uefi-bootkit-myth-confirmed/.

[Wal26] John Walker. Playstation is deleting 551 movies from customers' accounts, reminding us nothing digital is ever truly ours, 2026. Online; accessed 17-August-2026. URL: https://kotaku.com/playstation-store-movies-digital-studio-canal-terminator-2000711013.